

The Same Layer in PyTorch, and on a GPU

PyTorch's arrays are called tensors, its notation is nearly NumPy's, and it can run the same line on a second processor.

From arrays to tensors

Name	Dimensions	Example in this meeting	Shape
scalar	0	the bias of one neuron	a single number
vector	1	one pixel; the weights of one neuron	(3,)
matrix	2	the points X ; the weights W of a layer	(n , 3) and (K , 3)
volume	3	a colour image	(height, width, 3)
(no special name)	4	a batch of colour images	(30, 96, 96, 3)

- All of them are **tensors**. In mathematics, tensors with more than two dimensions have a theory of their own. In this course a tensor is a data structure: an array of numbers with some number of dimensions.
- PyTorch calls its arrays tensors. `torch.from_numpy` makes one from a NumPy array, `torch.tensor` from a list.

Three tensors

```
w_t = torch.tensor([0.299, 0.587, 0.114])    # the weights of the grey neuron, from a list
X_t = torch.from_numpy(X)                   # the points, from the NumPy array X
image_t = torch.from_numpy(image)           # the image, from the NumPy array image
```

```
w_t      dimensions: 1    shape: (3,)    type of its numbers: torch.float32
X_t      dimensions: 2    shape: (320640, 3)  type of its numbers: torch.float64
image_t  dimensions: 3    shape: (501, 640, 3)  type of its numbers: torch.float64
```

- `w_t` was made by PyTorch: its numbers are `float32`, 4 bytes each, about 7 significant digits.
`X_t` kept the type of the NumPy array, `float64`: 8 bytes, about 16 digits.
- `float32` is enough for neural networks, it takes half the memory, and a GPU computes fastest with it.

A first difference: the type of the numbers

```
try:
    X_t @ w_t
except RuntimeError as error:
    print("X_t @ w_t fails:", error)

X_t = X_t.float() # the same numbers, as float32
megabytes = X_t.element_size() * X_t.nelement() / 1e6
print("after X_t = X_t.float(), the type of its numbers is", X_t.dtype)
print(f"memory of the points: {X_t.nbytes / 1e6:.1f} MB as float64, {megabytes:.1f} MB as float32")

s_t = X_t @ w_t # one neuron, many points: the line of Act 1
```

```
X_t @ w_t fails: addmmv input tensors must have the same dtype, but got Float, Double, and Float
after X_t = X_t.float(), the type of its numbers is torch.float32
memory of the points: 7.7 MB as float64, 3.8 MB as float32
shape of X_t @ w_t: (320640,)
```

PyTorch does not multiply tensors of two different types. `.float()` converts to `float32`, and the points take half the memory.

The layer of Act 1, in PyTorch

```
W_t = torch.from_numpy(W).float()      # the weights of the layer, as a float32 tensor
b_t = torch.from_numpy(b).float()      # the biases of the layer

S_t = X_t @ W_t.T + b_t                # the line of Act 1, unchanged
print("shape of S_t:", tuple(S_t.shape))
print(f"largest difference from the NumPy scores S: {np.abs(S_t.numpy() - S).max():.1e}")
```

```
shape of S_t: (320640, 5)
largest difference from the NumPy scores S: 1.8e-07
```

- The line `X @ W.T + b` needs no change: PyTorch has `@`, `.T` and broadcasting, with the meaning they have in NumPy.
- The scores agree with NumPy's to about seven digits, not exactly: float32.

NumPy and PyTorch, side by side

	NumPy	PyTorch
make an array from a list	<code>np.array([1.0, 2.0])</code>	<code>torch.tensor([1.0, 2.0])</code>
the layer	<code>X @ W.T + b</code>	<code>X @ W.T + b</code>
a function of every entry	<code>np.exp(S)</code>	<code>torch.exp(S)</code>
along the rows	<code>axis=1, keepdims=True</code>	<code>dim=1, keepdim=True</code> ; NumPy's names are accepted too
the largest entry of each row	<code>S.max(axis=1)</code> is an array	<code>S.max(dim=1)</code> holds <code>.values</code> and <code>.indices</code>
decimal numbers	<code>float64</code>	<code>float32</code>
where the numbers are	in the memory of the CPU	on a device : the CPU or a GPU

The softmax, translated

```
def softmax_rows(S):  
    shifted = S - S.max(axis=1, keepdims=True)  
    E = np.exp(shifted)  
    return E / E.sum(axis=1, keepdims=True)
```

```
def softmax_rows_torch(S):  
    shifted = S - S.max(dim=1, keepdim=True).values  
    E = torch.exp(shifted)  
    return E / E.sum(dim=1, keepdim=True)
```

- An AI assistant translated the NumPy function to stated requirements: a tensor in, a tensor out, no loop, no NumPy, and not PyTorch's own softmax.
- Three differences: `torch.exp` for `np.exp`; `dim` and `keepdim` for `axis` and `keepdims`; and `.values` after `S.max(dim=1, keepdim=True)`, which returns the largest values together with their positions.
- No NumPy inside: NumPy cannot compute with a tensor that is on a GPU.

PyTorch's own layer

```
linear = torch.nn.Linear(3, K)           # a layer of K neurons for points with 3 coordinates
print("shape of linear.weight:", tuple(linear.weight.shape))
print("shape of linear.bias: ", tuple(linear.bias.shape))

with torch.no_grad():                    # this cell needs no gradients
    linear.weight.copy_(W_t)              # the weights of Act 1 go into the layer
    linear.bias.copy_(b_t)                # and so do the biases
    S_linear = linear(X_t)                # the layer, applied to all the points
```

```
shape of linear(X_t): (320640, 5)
largest difference between linear(X_t) and X_t @ W_t.T + b_t: 0.0e+00
```

- A layer of K neurons for points with 3 coordinates, with random weights and biases. Its weights have shape (K, 3), **one neuron in each row**; the documentation writes $y = xA^T + b$: the line of Act 1.
- Training adjusts the weights in many small steps, and each step needs to know how a change of each weight would change the answers: PyTorch computes these numbers, called gradients. `torch.no_grad()` says that this cell needs none.

A second processor: the GPU

- The processor that has run every program of this course so far is the **CPU**, the central processing unit: a few **cores**, each able to run any program, one instruction after another, very fast.
- A **GPU**, a graphics processing unit, was built for one kind of work: the colours of the pixels of a screen, many times a second. One short computation, repeated for millions of pixels that do not depend on each other. So: **thousands of small cores** that carry out the same instructions on different numbers at once, and **a memory of its own**. The T4 that Colab lends has 2,560 cores and 16 GB.
- A product of matrices is work of the same kind: each entry of the result is a sum of products that depends on no other entry. That is why neural networks are computed on GPUs.

The page **CPU and GPU**, parts 1 and 2, shows the two processors side by side:
ikoutis.github.io/cs301-labs/unit-2/pages/cpu-and-gpu.html

A tensor lives on a device

```
if GPU:
    X_g = X_t.to("cuda")           # copies of the three tensors, in the memory of the GPU
    W_g = W_t.to("cuda")
    b_g = b_t.to("cuda")
    print("X_g is on the device:", X_g.device)

    try:
        X_g @ W_t.T                # X_g is on the GPU and W_t is on the CPU
    except RuntimeError as error:
        print("X_g @ W_t.T fails:", error)

    S_g = X_g @ W_g.T + b_g        # the line of Act 1, carried out by the GPU
    print("S_g is on the device:", S_g.device)
```

- `X_t.to("cuda")` **copies** the numbers of a tensor into the memory of the GPU and returns the copy. PyTorch names the GPU `cuda`, after the system with which NVIDIA's GPUs are programmed.
- An operation needs **all of its tensors on one device**, and that device's processor carries it out: `X_g @ W_t.T` fails. `.cpu()` copies a result back. The line of the scores does not change.

Measure the product on both processors

```
def time_product(size, device):  
    """The average time, in seconds, of  $X @ W.T + b$  on the device, with  $n = m = K = \text{size}$ ."""  
    X = torch.rand(size, size, device=device)          # random float32 numbers, made on the device  
    W = torch.rand(size, size, device=device)  
    b = torch.rand(size, device=device)  
    repeats = max(1, min(200, 8_000_000_000 // size ** 3))  # many repeats when the product is small  
    finish(device)  
    start = time.perf_counter()  
    for _ in range(repeats):  
        S = X @ W.T + b  
    finish(device)  
    return (time.perf_counter() - start) / repeats
```

- `n`, `m` and `K` all equal to `size`, random numbers made on the device, many repeats when the product is small.
- Two details particular to a GPU: PyTorch hands the work to the GPU and goes on at once, so `finish` waits with `torch.cuda.synchronize()` before the clock is read; and the first use of a device is slow, so one small product is run first and not measured.

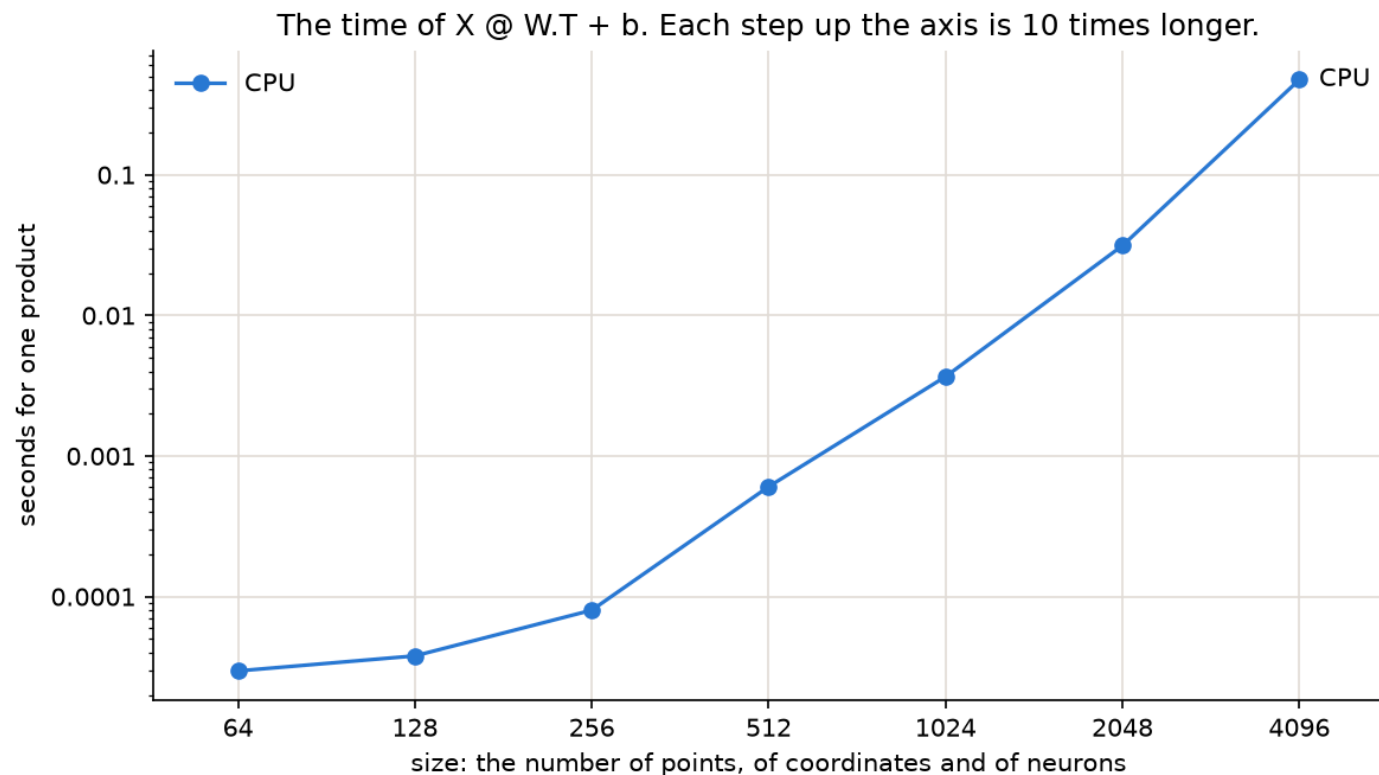
The time of the product on a CPU

`X @ W.T + b` with `n = m = K = size`, in seconds. CPU: 32 logical cores. GPU: none.

size	multiplications	CPU
64	262,144	0.000030
128	2,097,152	0.000038
256	16,777,216	0.000080
512	134,217,728	0.000605
1024	1,073,741,824	0.003667
2048	8,589,934,592	0.031320
4096	68,719,476,736	0.474551

- Measured on the machine that built these slides, which has no GPU: **the GPU column is what you measure in Colab.**
- From 2,048 to 4,096 the size doubles and the time grows by a factor near 8. Act 3 explains the factor.
- **Guess before you measure:** at which sizes will the GPU be faster than the CPU?

The same numbers as a chart



Both axes are stretched: each size is twice the one before, and each mark on the vertical axis is ten times the one below. A straight line means the same factor for each doubling. Your chart will have a second line.

Now in Colab: Act 2

- **Act 2** of the class notebook: the layer in PyTorch, and, if your session has a GPU, the same line carried out by the GPU.
- Run the measurement: seven sizes on the CPU and on the GPU, up to half a minute. From the table: at which sizes is the CPU the faster one? By what factor does the CPU's time grow from 2,048 to 4,096?
- **Post**, as a reply to your group's first post: the printed line, the chart with one sentence on what it shows, your two answers, and the name of your GPU. One member posts; begin with the names of all the members.